

Sujet examen blanc LEPL1110

Equipe pédagogique — Avril 2026

1 Allongement d'une tige sous son propre poids

Une tige métallique modélisée par un milieu 1D est accrochée à un plafond. La tige s'allonge sous son propre poids. La barre a une densité ρ , une raideur E et une longueur initiale l . L'énergie potentielle du système s'écrit comme

$$\Pi(u) = \int_0^l \frac{1}{2}(u')^2 - au \, dx \quad (1)$$

La fonction inconnue $u(x)$ donne le déplacement vers le bas en chaque point x , où x est la distance au plafond. La tige est accrochée au plafond. On impose donc $u(0) = 0$. La notation u' indique la dérivée de u par rapport à x et a est un nombre sans dimension donné par $a = \rho gl/E$ où g est la gravité.

1.1 Recherche de la solution par Rayleigh-Ritz

On propose l'approximation

$$u^r(x) = \alpha_1 x + \alpha_2 x^2 \quad (2)$$

Chercher les coefficients α_1, α_2 minimisant l'énergie potentielle.

L'approximation proposée ne comporte que deux termes. Pouvez-vous donner un exemple de troisième terme ?

Solution

$$u_r(x) = \alpha_1 x + \alpha_2 x^2, \quad u'_r(x) = \alpha_1 + 2\alpha_2 x.$$

$$\Pi(u_r) = \int_0^1 \left[\frac{1}{2} (u'_r)^2 - a u_r \right] dx = \int_0^1 \left[\frac{1}{2} (\alpha_1 + 2\alpha_2 x)^2 - a(\alpha_1 x + \alpha_2 x^2) \right] dx.$$

$$\Pi(\alpha_1, \alpha_2) = \frac{1}{2} \alpha_1^2 + \alpha_1 \alpha_2 + \frac{2}{3} \alpha_2^2 - \frac{a}{2} \alpha_1 - \frac{a}{3} \alpha_2.$$

$$\frac{\partial \Pi}{\partial \alpha_1} = \alpha_1 + \alpha_2 - \frac{a}{2} = 0, \quad \frac{\partial \Pi}{\partial \alpha_2} = \alpha_1 + \frac{4}{3} \alpha_2 - \frac{a}{3} = 0.$$

$$\alpha_2 = -\frac{a}{2}, \quad \alpha_1 = a.$$

$$u_r(x) = ax - \frac{a}{2} x^2.$$

Solution $\alpha_3 x^3$ ou plus généralement $\alpha_3 f(x)$ où f est une fonction continue nulle en $x=0$.

1.2 Comparaison avec la solution analytique

Vérifier si la solution approchée u^r est la solution exacte. Faites cela en deux temps.

- Trouver l'équation différentielle ordinaire que doit vérifier u . Pour rappel, cette équation est obtenue en utilisant la condition

$$\frac{d}{d\epsilon} \Pi(u + \epsilon v) |_{\epsilon=0} = 0 \quad \forall v \in V_0 = \{v \in H^1(0,1), v(0) = 0\} \quad (3)$$

et en éliminant v par intégration par partie.

- Vérifier si u^r satisfait cette équation différentielle ordinaire.

- Dans l'énoncé il est écrit " $v \in H^1(0,1)$ ". Expliquez ce que veut dire le symbole $H^1(0,1)$; peut-on dériver v et si oui comment ? Expliquez brièvement.

Solution

$$\left. \frac{d}{d\varepsilon} \Pi(u + \varepsilon v) \right|_{\varepsilon=0} = \int_0^1 (u'v' - av) dx = 0 \quad \forall v \in V_0, v(0) = 0.$$

$$\int_0^1 u'v' dx = [u'v]_0^1 - \int_0^1 u''v dx.$$

$$u'(1)v(1) - \int_0^1 (u'' + a)v dx = 0 \quad \forall v.$$

$$\boxed{u''(x) + a = 0 \text{ dans } (0,1),} \quad \boxed{u'(1) = 0,} \quad \boxed{u(0) = 0.}$$

Le maximum des points est obtenu en donnant $u''(x) + a = 0$ et un bonus de 1 point est donné si la condition $u'(1) = 0$ est également fournie.

$$u'(x) = -ax + C_1, \quad u'(1) = 0 \Rightarrow C_1 = a.$$

$$u'(x) = a(1 - x).$$

$$u(x) = ax - \frac{a}{2}x^2 + C_2, \quad u(0) = 0 \Rightarrow C_2 = 0.$$

$$\boxed{u(x) = ax - \frac{a}{2}x^2.}$$

$$u_r(x) = u(x).$$

La solution approchée de Rayleigh-Ritz u_r est donc égale à la solution analytique.

Solution. $H^1(0,1)$ réfère à l'espace de Sobolev d'ordre 1 sur l'intervalle $(0,1)$. Il est défini comme l'ensemble des fonctions $u \in L^2(0,1)$ qui admettent une dérivée faible dans $L^2(0,1)$. On peut donc dériver $v \in H^1(0,1)$ mais au sens faible qui n'est pas le sens classique appris dans les cours d'analyse introductifs. Comme nous sommes en dimension 1, v est uniformément continu sur $[0,1]$.

1.3 Solution élément fini

On discrétise la tige avec deux éléments finis. Le premier élément fini couvre le segment $[x_0, x_1]$ et le second, le segment $[x_1, x_2]$ avec $x_0 = 0$, $x_1 \in (0, 1)$ et $x_2 = 1$.

- Ecrire le système linéaire que doit vérifier la solution élément fini u_1, u_2 aux noeuds x_1 et x_2 .

- Trouver la solution élément fini dans le cas $x_1 = 1/2$.

- Cette solution élément finis est-elle nodalement exacte ?

Solution

$$x_0 = 0, \quad x_1 \in (0, 1), \quad x_2 = 1.$$

On note

$$h_1 = x_1, \quad h_2 = 1 - x_1.$$

Pour un élément linéaire 1D de longueur h_e , la matrice de rigidité locale est

$$K^{(e)} = \frac{1}{h_e} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix},$$

et le vecteur de charge local est

$$f^{(e)} = \frac{ah_e}{2} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Ainsi, pour les deux éléments, on a

$$K^{(1)} = \frac{1}{h_1} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \quad K^{(2)} = \frac{1}{h_2} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix},$$

$$f^{(1)} = \frac{ah_1}{2} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad f^{(2)} = \frac{ah_2}{2} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Après assemblage, le système global s'écrit

$$\begin{pmatrix} \frac{1}{h_1} & -\frac{1}{h_1} & 0 \\ -\frac{1}{h_1} & \frac{1}{h_1} + \frac{1}{h_2} & -\frac{1}{h_2} \\ 0 & -\frac{1}{h_2} & \frac{1}{h_2} \end{pmatrix} \begin{pmatrix} u_0 \\ u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} \frac{ah_1}{2} \\ \frac{a(h_1+h_2)}{2} \\ \frac{ah_2}{2} \end{pmatrix}.$$

Comme $u_0 = 0$, le système réduit devient

$$\begin{pmatrix} \frac{1}{h_1} + \frac{1}{h_2} & -\frac{1}{h_2} \\ -\frac{1}{h_2} & \frac{1}{h_2} \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} \frac{a(h_1+h_2)}{2} \\ \frac{ah_2}{2} \end{pmatrix}.$$

Comme $h_1 + h_2 = 1$, on obtient

$$\begin{pmatrix} \frac{1}{x_1} + \frac{1}{1-x_1} & -\frac{1}{1-x_1} \\ -\frac{1}{1-x_1} & \frac{1}{1-x_1} \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} \frac{a}{2} \\ \frac{a(1-x_1)}{2} \end{pmatrix}.$$

Considérons maintenant le cas

$$x_1 = \frac{1}{2}.$$

Alors

$$h_1 = h_2 = \frac{1}{2},$$

et le système réduit devient

$$\begin{pmatrix} 4 & -2 \\ -2 & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} \frac{a}{2} \\ \frac{a}{4} \end{pmatrix}.$$

La solution exacte est

$$u(x) = ax - \frac{a}{2}x^2.$$

Aux nœuds $x_1 = \frac{1}{2}$ et $x_2 = 1$, on a

$$u\left(\frac{1}{2}\right) = a \cdot \frac{1}{2} - \frac{a}{2} \left(\frac{1}{2}\right)^2 = \frac{a}{2} - \frac{a}{8} = \frac{3a}{8}, \quad u(1) = a - \frac{a}{2} = \frac{a}{2}.$$

On teste donc

$$u_1 = \frac{3a}{8}, \quad u_2 = \frac{a}{2}.$$

Par substitution :

$$\begin{pmatrix} 4 & -2 \\ -2 & 2 \end{pmatrix} \begin{pmatrix} \frac{3a}{8} \\ \frac{a}{2} \end{pmatrix} = \begin{pmatrix} 4 \cdot \frac{3a}{8} - 2 \cdot \frac{a}{2} \\ -2 \cdot \frac{3a}{8} + 2 \cdot \frac{a}{2} \end{pmatrix} = \begin{pmatrix} \frac{3a}{2} - a \\ -\frac{3a}{4} + a \end{pmatrix} = \begin{pmatrix} \frac{a}{2} \\ \frac{a}{4} \end{pmatrix}.$$

Donc la solution exacte vérifie le système nodal lorsque $x_1 = \frac{1}{2}$. Par conséquent, dans ce cas,

$$\boxed{u_h\left(\frac{1}{2}\right) = u\left(\frac{1}{2}\right) = \frac{3a}{8}, \quad u_h(1) = u(1) = \frac{a}{2}.$$

Ainsi, pour $x_1 = \frac{1}{2}$, la solution éléments finis est nodalement exacte.

2 Le jeu des 7 erreurs

La formulation par éléments finis de la barre soumise à son propre poids a été implémentée en Python dans le code ci-dessous. On suppose une barre de taille 1, comme dans la question 1. Nous allons discrétiser cette barre avec N éléments finis P^1 en 1D.

Nous utilisons 2 points d'intégration dont la position et les poids sont donnés explicitement dans le code.

Le code retourne la matrice de raideur K et le membre de droite f . Les conditions aux limites ne sont pas implémentées – on fait cela dans une autre fonction non implémentée ici.

Ce code contient volontairement *un maximum de sept erreurs*.

Repérez ces erreurs en annotant le code à l'aide des numéros ① à (maximum) ⑦. Pour chacune d'elles, reportez-vous à la liste ci-dessous et fournissez un bref commentaire (une phrase maximum) expliquant la nature de l'erreur.

①	
②	
③	
④	
⑤	
⑥	
⑦	

```

import numpy as np

def assemble_bar_1d(N, a):
    x = np.linspace(0.0, 1.0, N)
    K = np.zeros((N + 1, N + 1))
    f = np.zeros(N + 1)

    xi_q = [-1.0 / np.sqrt(3.0), 1.0 / np.sqrt(3.0)]
    w_q = [1.0, 1.0]

    dN_dxi = [-0.5, 0.5]

    for e in range(N):
        x0 = x[e]
        x1 = x[e+1]
        h = x1 - x0
        Jac = h
        dxi_dx = 1.0 / Jac

        dof = [e, e+1]

        for q in range(2):
            xi = xi_q[q]
            w = w_q[q]

            Nq = [0.0, 0.0]
            Nq[0] = 0.5 * (1.0 - xi)
            Nq[1] = 0.5 * (1.0 + xi)

            dN_dx = [0.0, 0.0]
            for i in range(2):
                dN_dx[i] = dN_dxi[i] / dxi_dx

            for i in range(2):
                I = dof[i]

                f[I] = f[I] + Nq[i] * Jac

                for j in range(2):
                    J = dof[j]

                    K[I, J] = K[I, J] + dN_dx[i] * dN_dx[j] * w * Jac

    return K, f

```

Solution

- ① `np.linspace(0.0, 1.0, N)` crée seulement N noeuds, alors qu'il en faut $N + 1$ pour N éléments.
- ② Le jacobien de la transformation de $[-1, 1]$ vers un élément de longueur h vaut $h / 2.0$, pas h .
- ③ La dérivée physique doit être calculée par $dN/dx = (dN/d\xi)(d\xi/dx)$, donc il faut multiplier par `dxi_dx`.
- ④ f doit contenir le facteur a .

- ⑤ f doit aussi contenir le poids d'intégration w .
- ⑥ Dans l'assemblage de la matrice, l'indice global de colonne doit être `dof[j]` et non `dof[i]`.

Il n'y a donc que six erreurs effectives dans ce code, malgré les sept emplacements proposés.

```
import numpy as np

def assemble_bar_1d(N, a):
    x = np.linspace(0.0, 1.0, N + 1)
    K = np.zeros((N + 1, N + 1))
    f = np.zeros(N + 1)

    xi_q = [-1.0 / np.sqrt(3.0), 1.0 / np.sqrt(3.0)]
    w_q = [1.0, 1.0]

    dN_dxi = [-0.5, 0.5]

    for e in range(N):
        x0 = x[e]
        x1 = x[e+1]
        h = x1 - x0
        Jac = h / 2.0
        dxi_dx = 1.0 / Jac

        dof = [e, e+1]

        for q in range(2):
            xi = xi_q[q]
            w = w_q[q]

            Nq = [0.0, 0.0]
            Nq[0] = 0.5 * (1.0 - xi)
            Nq[1] = 0.5 * (1.0 + xi)

            dN_dx = [0.0, 0.0]
            for i in range(2):
                dN_dx[i] = dN_dxi[i] * dxi_dx

            for i in range(2):
                I = dof[i]

                f[I] = f[I] + a * Nq[i] * w * Jac

            for j in range(2):
                J = dof[j]

                K[I, J] = K[I, J] + dN_dx[i] * dN_dx[j] * w * Jac

    return K, f
```